# Solution: Implement Trie (Prefix Tree)

## Problem Statement

Design and implement a Trie (also known as a Prefix Tree). A trie is a tree-like data structure that stores a dynamic set of strings, and is particularly useful for searching for words with a given prefix.

Implement the `Solution` class:

- `Solution()` Initializes the object.
- `void insert(word)` Inserts `word` into the trie, making it available for future searches.
- `bool search(word)` Checks if the word exists in the trie.
- `bool startsWith(word)` Checks if any word in the trie starts with the given prefix.

## Examples

1. **Example 1:**
   - **Input**:
     - Trie operations: `["Trie", "insert", "search", "startsWith"]`
     - Arguments: `[[], ["apple"], ["apple"], ["app"]]`
   - **Expected Output**: `[-1, -1, 1, 1]`
   - **Justification**: After inserting "apple", "apple" exists in the Trie. There is also a word that starts with "app", which is "apple".
2. **Example 2:**
   - **Input**:
     - Trie operations: `["Trie", "insert", "search", "startsWith", "search"]`
     - Arguments: `[[], ["banana"], ["apple"], ["ban"], ["banana"]]`
   - **Expected Output**: `[-1, -1, 0, 1, 1]`
   - **Justification**: After inserting "banana", "apple" does not exist in the Trie but a word that starts with "ban", which is "banana", does exist.
3. **Example 3:**
   - **Input**:
     - Trie operations: `["Trie", "insert", "search", "search", "startsWith"]`
     - Arguments: `[[], ["grape"], ["grape"], ["grap"], ["gr"]]`
   - **Expected Output**: `[-1, -1, 1, 1, 1]`
   - **Justification**: After inserting "grape", "grape" exists in the Trie. There are words that start with "grap" and "gr", which is "grape".

## Solution

The trie is represented as a tree, where each node contains an array of pointers (or references) to its children and a boolean flag indicating if the current node marks the end of a word. When inserting or searching for a word, we start at the root node and navigate through the tree character by character until we either finish the operation or determine the word doesn't exist in the trie.

Now, let's break down the operations:

1. **Insert**:
   - We begin at the root node.
   - For every character in the word, check if there's a child node for it.
   - If the child node doesn't exist, we create it.
   - Navigate to the child node and repeat the process for the next character.
   - Once the end of the word is reached, mark the current node as an endpoint of a word.
2. **Search**:
   - Starting at the root, traverse the trie character by character.
   - For every character in the word, check if there's a child node for it.
   - If at any point there isn't a child node for the character, the word doesn't exist in the trie.
   - If we can traverse the entire word and the last node is marked as an endpoint, the word exists in the trie.
3. **StartsWith**:
   - The operation is similar to the search, but we don't need the last node to be an endpoint.
   - If we can traverse the prefix without any missing nodes, there exists a word in the trie that starts with the given prefix.

## Algorithm Walkthrough

Using Example 1:

- `["Trie", "insert", "search", "startsWith"]`
- `[[], ["apple"], ["apple"], ["app"]]`

1. Create an empty Trie.
2. Insert "apple".
   - Start from the root. For 'a', move to the child node or create one if it doesn't exist.
   - Move to 'p', then the next 'p', followed by 'l' and finally 'e'. Mark 'e' as the end of a word.
3. Search for "apple".
   - Start from the root and traverse nodes 'a' -> 'p' -> 'p' -> 'l' -> 'e'. Since 'e' is marked as the end of a word, return true.
4. Check if a word starts with "app".
   - Traverse nodes for 'a' -> 'p' -> 'p'. All nodes exist, so return true.

## Code

Java

```java
// class TrieNode {
//     TrieNode[] children = new TrieNode[26]; // Represents each letter of the alphabet
//     boolean isEnd; // Flag to represent if the node is the end of a word.

//     public TrieNode() {}
// }

public class Solution {
    private TrieNode root;

    public Solution() {
        root = new TrieNode();
    }

    // Inserts a word into the trie.
    public void insert(String word) {
        TrieNode node = root;
        for (char c : word.toCharArray()) {
            if (node.children[c - 'a'] == null) {
                node.children[c - 'a'] = new TrieNode();
            }
```

Run Code Result Stdout

You must run your code first

Execute

## Complexity Analysis

- **Time Complexity:**
  - Insert: (O(m)), where (m) is the key length.
  - Search and StartsWith: (O(m)) in the worst case scenario.
- **Space Complexity:** (O(n \times m)), where (n) is the number of inserted keys and (m) is the average key length.

← Previous

Implement Trie (Prefix Tree) (Medium)

Next →

Design Add And Search Words Data Structure (Medium)

☑ Mark as Completed